Titulación: **Grado en Matemáticas y Computación**
Curso: **2025-2026. Convocatoria Ordinaria de Junio**
Asignatura: **Bases de Datos Avanzadas – Laboratorio**

Practica 3: Seguridad, Usuarios y Transacciones.

ALUMNO 1:
Nombre y Apellidos: José Antonio López González
DNI: 03209889D

ALUMNO 2:
Nombre y Apellidos: Jorge López Duque
DNI: _____

Fecha: 17/05/2026

Profesor Responsable: Javier Arrollo Gallardo

Mediante la entrega de este fichero los alumnos aseguran que cumplen con la normativa de autoría de trabajos de la Universidad de Alcalá, y declaran éste como un trabajo original y propio.

En caso de ser detectada copia, se puntuará **TODA** la asignatura como Suspenso – Cero.

Plazos

Tarea en laboratorio:	Semana 20 de abril, Semana 27 de abril, Semana 4 de mayo y semana 11 de mayo.
Entrega de práctica:	Día 17 de mayo. Aula Virtual
Documento a entregar:	Este mismo fichero con las respuestas a las cuestiones planteadas, con el código SQL utilizado en cada uno de los apartados. Así mismo se debe de entregar el fichero de configuración postgresql.conf usado en la práctica, así como los ficheros de log de postgres (obligatorios). Se entregará en un ZIP comprimido: **`DNI'sdelosAlumnos_PECL3.zip`**

Introducción

El contenido de esta práctica versa sobre el manejo de las transacciones en sistemas de bases de datos, así como el control de la concurrencia y la recuperación de la base de datos frente a una caída del sistema. Las transacciones se definen como una unidad lógica de procesamiento compuesta por una serie de operaciones simples que se ejecutan como una sola operación. Entre las etiquetas BEGIN y COMMIT del lenguaje SQL se insertan las operaciones simples a realizar en una transacción. La sentencia ROLLBACK sirve para deshacer todos los cambios involucrados en una transacción y devolver a la base de datos al estado consistente en el que estaba antes de procesar la transacción. También se verá el registro diario o registro histórico del sistema de la base de datos (en PostgreSQL se denomina WAL: Write Ahead Loggin) donde se reflejan todas las operaciones sobre la base de datos y que sirve para recuperar ésta a un estado consistente si se produjera un error lógico o de hardware. La versión de postgres a utilizar deberá ser la versión 17 preferentemente.

Actividades y Cuestiones

En esta parte la base de datos **MUSICOS** deberá de ser nueva y no contener datos. Además, consta de 5 actividades:

- Conceptos generales.
- Manejo de transacciones.
- Concurrencia.
- Registro histórico.
- Backup y Recuperación

Cuestión 0: Configurar el fichero de Error Reporting and Logging de PostgreSQL para que aparezcan recogidas las sentencias SQL DDL (Lenguaje de Definición de Datos) + DML (Lenguaje de Manipulación de Datos) generadas en dicho fichero. No se pide activar todas las sentencias. No activar la duración de la consulta. También se debe de configurar el log para que en el comienzo de la línea de registro de la información del log ("line prefix") aparezca el DNI de los alumnos que realizan la práctica (ambos), el nombre del host con su puerto, y la fecha y hora de la operación que se ha realizado.

Ponemos:
logging_collector = on (como estaba)
log_statement = 'mod' (como estaba)
log_duration = off (como estaba)
log_line_prefix = '70943373B_03209889D | %r | %t | ' (ligera modificacion con respecto a la practica anterior).

Acordándonos de descomentar lo neceario.

Cuestión 1: ¿qué es el WAL en postgres? Si no está activo, activarlo. Determinar cuál es el directorio y el archivo donde se guarda. ¿Cuál es su tamaño? Al abrir el archivo con un editor de textos, ¿se puede deducir algo de lo que guarda el archivo?

El WAL (Write-Ahead Logging) es un mecanismo de registro que utiliza PostgreSQL para garantizar la durabilidad y recuperación de la base de datos.

Consiste en escribir primero en un registro (log) todos los cambios que se van a realizar sobre la base de datos antes de aplicarlos físicamente en disco.
De esta forma, si ocurre un fallo (por ejemplo, caída del sistema), PostgreSQL puede reconstruir el estado consistente de la base de datos utilizando dicho registro.

Esta dentro de Postgre en data/pg_wall, el tamaño son unos 16 megas.

Aunque el archivo WAL no es legible directamente al estar en formato binario, al abrirlo con un editor de texto se pueden observar fragmentos de información reconocible (como nombres o valores insertados en la base de datos).
Sin embargo, estos datos aparecen desordenados y mezclados con información interna del sistema, por lo que no es posible interpretar su contenido de forma directa sin herramientas específicas.

Cuestión 2: Abrir una transacción que realice una operación de inserción de un grupo y un músico que pertenece a ese grupo en la base de datos **MUSICOS**. Cerrar la transacción con éxito. Entonces abrir el archivo de diario ¿Se encuentra reflejada la operación en el archivo del sistema? ¿Por qué?

Sí, la operación se encuentra reflejada en el archivo de diario (WAL), ya que PostgreSQL registra todas las modificaciones en este fichero antes de aplicarlas en la base de datos.

Esto se debe al mecanismo Write-Ahead Logging, que garantiza la durabilidad de las transacciones.

Al finalizar la transacción con COMMIT, los cambios quedan confirmados y pueden ser recuperados en caso de fallo del sistema.

-Codigo:

```
BEGIN;

SELECT txid_current();

INSERT INTO Grupo VALUES
(1, 'Grupo1', 'Rock', 'España', 'www.grupo1.com');
```

Cuestión 3: Aplicar el comando pg_waldump.exe al último fichero de WAL que se haya generado. Obtener las estadísticas de ese fichero y comentar qué se está viendo.

La herramienta pg_waldump no puede ejecutarse desde DataGrip, ya que no es una instrucción SQL, sino una utilidad del sistema de PostgreSQL.
Por ello, se ha ejecutado desde la consola de Windows (CMD), accediendo al directorio de instalación de PostgreSQL.

Aplicando "D:\Postgre\PostgreSQL\18\bin\pg_waldump.exe" --stats 0 000000010000002F000000D7
Observamos una tabla con las siguientes filas:
Heap = Operaciones sobre tablas (INSERT, UPDATE, DELETE)
Btree = Operaciones sobre índices (índices de PK y FK)
Transaction = Eventos de transaccion (BEGIN, COMMIT, ROLLBACK)
XLOG = Registros internos del sistema (Control de WAL, metadatos del sistema)
Entre otras.

Y las siguientes columnas:
N (%): Número de registros de ese tipo (Cuántas veces aparece)
Record size (%): Tamaño de los registros lógicos (Cuánta información "útil" generan)
FPI size (%): Copias completas de páginas de datos (Se usan para recuperación tras fallos)
Combined size (%): Record size + FPI size (Es el impacto total en el WAL)

Cuestión 4: Determinar el identificador de la transacción que realizó la operación de inserción de la cuestión 2. Aplicar el comando anterior al último fichero de WAL que se ha generado y mostrar los registros que se han creado para esa transacción. ¿Qué se puede ver? Interpretar los resultados obtenidos.

Ejecutamos: "D:\Postgre\PostgreSQL\18\bin\pg_waldump.exe" 000000010000002F000000D7 > wal.txt y abrimos wal.txt

El identificador de la transacción que realizó las inserciones es el 1594 (en el momento de ejecucion), obtenido mediante la función txid_current().

El análisis del fichero WAL muestra todos los registros asociados a dicha transacción, comenzando con las operaciones de inserción en las tablas (Heap INSERT), seguidas de la actualización de índices primarios y foráneos (Btree), así como mecanismos de control de concurrencia mediante bloqueos (LOCK).

Finalmente, la transacción concluye con un registro de COMMIT, lo que garantiza la persistencia de todos los cambios realizados.

Esto demuestra que PostgreSQL registra de forma secuencial y detallada todas las operaciones de una transacción en el WAL, permitiendo su recuperación y asegurando la consistencia de la base de datos.

Cuestión 5: Se va a crear un backup del clúster de postgres. Este backup será utilizado más adelante para recuperar el sistema frente a una caída del sistema. Realizar solamente el backup mediante el procedimiento descrito en el apartado 25.3 del manual (versión 18 es *"Continous Archiving and point-in-time recovery (PITR)"*. Comentar el procedimiento realizado.

Para realizar el backup del clúster se ha seguido el procedimiento de archivado continuo (PITR).

En primer lugar, se ha configurado el fichero postgresql.conf activando el modo de archivado (archive_mode = on) y definiendo el comando de copia de los archivos WAL mediante archive_command (archive_command = 'copy "%p" "D:/backup_wal/%f"').

Además, se ha establecido wal_level = replica para garantizar el registro completo de las operaciones. Posteriormente, se ha realizado un backup físico del clúster utilizando la herramienta pg_basebackup ("D:\Postgre\PostgreSQL\18\bin\pg_basebackup.exe" -U postgres -D "D:\backup_base" -Ft -z -P), que permite obtener una copia consistente de la base de datos en un momento determinado.

De forma simultánea, los archivos WAL generados se han almacenado en un directorio independiente, lo que permite reconstruir el estado de la base de datos en cualquier punto en el tiempo mediante técnicas de recuperación PITR.

Este procedimiento garantiza la posibilidad de recuperación ante fallos del sistema, combinando una copia base con el registro continuo de cambios en el WAL.

Cuestión 6: ¿Qué herramientas disponibles tiene PotsgreSQL para controlar la actividad de la base de datos en cuanto a la concurrencia y transacciones? ¿Qué información es capaz de mostrar? ¿Dónde se guarda dicha información? ¿Cómo se puede mostrar?

PostgreSQL dispone de diversas herramientas para controlar la actividad de la base de datos en cuanto a concurrencia y transacciones, principalmente a través de vistas del sistema como pg_stat_activity, pg_locks y pg_stat_database.

Estas vistas permiten obtener información sobre sesiones activas, consultas en ejecución, estados de las transacciones, bloqueos entre procesos y estadísticas de commits y rollbacks.
La información se almacena en la memoria interna del sistema y en los catálogos del sistema (pg_catalog), aunque también puede registrarse parcialmente en los ficheros de log.

Para consultar esta información se utilizan sentencias SQL, como SELECT * FROM pg_stat_activity o SELECT * FROM pg_locks, lo que permite monitorizar en tiempo real la concurrencia y detectar posibles problemas de bloqueo o rendimiento.

Cuestión 7: Crear tres usuarios en la base de datos que puedan acceder a la base de datos **MUSICOS** identificados como musico1, musico2 y musico3 que tengan permisos de lectura/escritura a la base de datos **MUSICOS**, pero que no puedan modificar su estructura. Describir el proceso seguido.

Se han creado tres usuarios (musico1, musico2 y musico3) mediante la sentencia CREATE USER, asignándoles una contraseña para su autenticación. (Ej: CREATE USER musico1 WITH PASSWORD 'pass1';)

Posteriormente, se les ha concedido acceso a la base de datos MUSICOS mediante el permiso CONNECT (GRANT CONNECT ON DATABASE musicos TO musico1, musico2, musico3;),
así como acceso al esquema public con USAGE (GRANT USAGE ON SCHEMA public TO musico1, musico2, musico3;).

Para permitir la lectura y escritura de datos, se han otorgado permisos SELECT, INSERT, UPDATE y DELETE sobre todas las tablas del esquema (GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO musico1, musico2, musico3;).

Además, se han configurado privilegios por defecto para que estos permisos se apliquen automáticamente a futuras tablas (ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO musico1, musico2, musico3;).

No se han concedido permisos de tipo CREATE, ALTER o DROP, por lo que los usuarios no pueden modificar la estructura de la base de datos, garantizando así la separación entre manipulación de datos y definición de la estructura.

-Codigo:
```
--Crear usuarios:
CREATE USER musico1 WITH PASSWORD 'pass1';
CREATE USER musico2 WITH PASSWORD 'pass2';
CREATE USER musico3 WITH PASSWORD 'pass3';

--Dar acceso a la base de datos:
GRANT CONNECT ON DATABASE postgres TO musico1, musico2, musico3;
```

Cuestión 8: Cierre la transacción anterior. Abrir una transacción T1 en el musico1 que realice las siguientes operaciones sobre la base de datos **MUSICOS**. NO termine la transacción. Simplemente:

- Inserte un nuevo grupo de música llamado "Piedras Negras" de género musical rock con código 1300.
- Inserte un músico del anterior grupo con DNI 123456789X de la ciudad de Santander en la provincia de Cantabria

Realizar cualquier consulta SQL que muestre los datos anteriores insertados para ver que todo está correcto. Consultar la información sobre lo que se encuentra relacionado con la transacción en la actividad del sistema. ¿Qué conclusiones se pueden extraer?

Se ha abierto una transacción T1 sin finalizarla, realizando inserciones en las tablas Grupo y Musicos.

Al consultar los datos desde la misma sesión, los cambios son visibles, ya que forman parte de la transacción activa.

Sin embargo, estos cambios aún no han sido confirmados mediante COMMIT, por lo que no seran visibles para otras transacciones.

El análisis de la actividad del sistema mediante pg_stat_activity muestra la transacción en estado activa, y pg_locks refleja los bloqueos asociados.

Se concluye que PostgreSQL aplica aislamiento de transacciones, permitiendo ver los cambios dentro de la propia transacción pero no desde otras sesiones hasta su confirmación.

-Codigo:

```
--Eliminacion de antigüos valores:
DELETE FROM Musicos where codigo_grupo_grupo = 1300;
DELETE FROM Grupo where codigo_grupo = 1300;

--Transaccion T1
BEGIN;

INSERT INTO Grupo VALUES
```

Cuestión 9: Establecer una **nueva conexión** a la base de datos con el musico2 (abrir otra sesión diferente a la abierta actualmente que pertenezca al musico2) y realizar la misma consulta. ¿Se nota algún cambio? En caso afirmativo, ¿a qué puede ser debido el diferente funcionamiento en la base de datos para ambas consultas? ¿Qué información de actividad hay registrada en la base de datos en este momento relacionadas con las transacciones abiertas?

Al realizar la misma consulta desde una nueva sesión con el usuario musico2, se observa que los datos insertados por musico1 no aparecen.

Este comportamiento se debe a que la transacción iniciada por musico1 no ha sido confirmada mediante COMMIT.

PostgreSQL utiliza por defecto el nivel de aislamiento READ COMMITTED, lo que implica que una transacción solo puede ver los cambios que han sido confirmados por otras transacciones.

Como consecuencia, los datos insertados permanecen invisibles para otras sesiones hasta que la transacción finaliza correctamente.

En cuanto a la actividad del sistema, en la vista pg_locks se pueden observar los bloqueos asociados a la transacción activa.

Esto demuestra el correcto funcionamiento del control de concurrencia y del aislamiento de transacciones en PostgreSQL, evitando inconsistencias como lecturas de datos no confirmados.

-Codigo:

Cuestión 10: ¿Se encuentran los nuevos datos físicamente en las tablas de la base de datos? Entonces, ¿de dónde se obtienen los datos de la cuestión 8 y/o de la 9?

Los nuevos datos no se encuentran físicamente en las tablas de la base de datos hasta que la transacción es confirmada mediante COMMIT.

Mientras la transacción permanece abierta, los cambios se almacenan en memoria (shared buffers) y en el registro WAL, pero no se escriben definitivamente en disco.

Por este motivo, en la cuestión 8 los datos son visibles únicamente para la transacción que los ha generado, mientras que en la cuestión 9 no son visibles desde otras sesiones.

Los datos consultados provienen del sistema de buffers internos de PostgreSQL en el caso de la sesión que realiza la transacción, y de las tablas físicas únicamente cuando la transacción ha sido confirmada.

Cuestión 11: Finalizar con éxito la transacción T1 y realizar la consulta de la cuestión 8 y 9 sobre ambos músicos conectados. ¿Qué es lo que se obtiene ahora? ¿Por qué?

Tras finalizar la transacción T1 mediante COMMIT, los datos insertados pasan a formar parte de la base de datos de forma permanente.

Al repetir las consultas desde ambas sesiones (musico1 y musico2), se observa que ahora los datos son visibles en ambos casos.

Esto se debe a que, una vez confirmada la transacción, PostgreSQL escribe los cambios de forma definitiva en las tablas físicas y los hace accesibles para todas las transacciones posteriores.

Antes del COMMIT, los datos solo eran visibles en la sesión que los había generado debido al aislamiento de transacciones, pero tras su confirmación pasan a formar parte de la base de datos.

Cuestión 12: Sin ninguna transacción en curso, abrir una transacción en un músico cualquiera y realizar las siguientes operaciones (en este orden):

- Insertar un nuevo grupo musical español con código 1400
- Insertar un nuevo músico que pertenezca al grupo anterior.
- Cambiar el teléfono del músico anterior a 918856931.
- Actualizar el código del grupo musical anterior a 1300.
- Cambiar el nombre del grupo musical anterior a “Héroes del Silencio”
- Cerrar la transacción.

¿Cuál es el estado final de la base de datos? ¿Por qué?

El estado final de la base de datos es el mismo que antes de iniciar la transacción, ya que no se ha aplicado ningún cambio.
Esto se debe a que durante la ejecución de la transacción se produce un error al intentar modificar el código del grupo, que es una clave primaria, por una ya existente.
Como consecuencia, PostgreSQL aborta automáticamente la transacción, impidiendo que se ejecuten las operaciones restantes y evitando la confirmación mediante COMMIT.
Este comportamiento refleja la propiedad de atomicidad de las transacciones, según la cual todas las operaciones deben ejecutarse correctamente para que los cambios sean aplicados; en caso contrario, ninguno de ellos se guarda, manteniendo la consistencia de la base de datos.

-Codigo:

```
--Transaccion T1
BEGIN;

-- 1. Insertar grupo
INSERT INTO Grupo VALUES
(1400, 'Metallica', 'rock', 'España', 'grupete.com');

-- 2. Insertar músico
INSERT INTO Musicos VALUES
(4001, '123456789Z', 'Manu', 'Santander', 39001, 'Santander', 'Cantabria', 600000000, 'Guitarra', 1400);
```

Cuestión 13: Repetir la cuestión 8 con otro grupo musical y otro músico que sean distintos. Realizar la misma consulta de la cuestión 9, pero ahora terminar la transacción con un ROLLBACK y repetir la consulta con los mismos dos músicos. ¿Cuál es el resultado? ¿Por qué?

Tras repetir el proceso de inserción en una nueva transacción, los datos son visibles únicamente desde la sesión que realiza la operación mientras la transacción permanece abierta, pero no desde otras sesiones.

Al finalizar la transacción mediante ROLLBACK, todos los cambios realizados son deshechos, por lo que los datos insertados desaparecen completamente (ROLLBACK elimina tanto los cambios visibles en la transacción como los registros asociados en el WAL no confirmados).

Al repetir la consulta desde ambas sesiones (musico1 y musico2), se observa que ninguno de los dos usuarios puede ver los datos, ya que estos nunca han sido confirmados en la base de datos.

Este comportamiento se debe a que ROLLBACK revierte todas las operaciones de la transacción, garantizando la propiedad de atomicidad y manteniendo la consistencia del sistema

Cuestión 14: Crear las siguientes tres tablas:

- ValorA con campo A de tipo real que sea PK.
- ValorB con campo B de tipo real que sea PK.
- ValorC con campo C de tipo real que sea PK.

Insertar una tupla en cada tabla con valor X=40, B=50 y C=60 respectivamente.

Se han creado tres tablas independientes (ValorA, ValorB y ValorC), cada una con una clave primaria de tipo REAL y se ha insertado un valor en cada tabla (40, 50 y 60 respectivamente).

-Codigo:

```
--Creacion de tablas:
CREATE TABLE ValorA (
   A REAL PRIMARY KEY
);

CREATE TABLE ValorB (
   B REAL PRIMARY KEY
);
```

Cuestión 15: Se suministra la tabla siguiente donde se muestra la planificación que hay que seguir por tres transacciones concurrentes asociadas cada una a un músico diferente que van a acceder en el orden marcado a las tablas anteriores. Se debe de abrir tres consolas SQL con cada uno de los usuarios (musico1, musico2 y musico3) y seguir el orden de cada instrucción teniendo en cuenta que un READ es una sentencia SELECT y un WRITE es una sentencia UPDATE. Rellenar la tabla con los valores que ven cada uno de los músicos en su ejecución de las instrucciones de cada transacción. Antes de la primera instrucción de cada transacción se debe ejecutar BEGIN;

Secuencia	T1 Musico1	T2 Musico2	T3 Musico3	Valores Musico1	Valores Musico2	Valores Musico3	Sentencia SQL
1	READ(C)	.	.	.	.	.	.
2	C=C/100	.	.	.	.	.	.
3	WRITE(C)	.	.	.	.	.	.
4	.	READ(B)	.	.	.	.	.
5	.	B=B+200	.	.	.	.	.
6	.	.	READ(A)	.	.	.	.
7	.	.	A=A*20	.	.	.	.
8	.	WRITE(B)	.	.	.	.	.
9	.	.	READ(B)	.	.	.	.
10	.	.	A=A-B	.	.	.	.
11	.	.	WRITE(A)	.	.	.	.
12	READ(A)	.	.	.	.	.	.
13	A=A+100	.	.	.	.	.	.
14	WRITE(A)	.	.	.	.	.	.
15	.	READ(C)	.	.	.	.	.
16	.	C=B-C	.	.	.	.	.
17	.	.	READ(C)	.	.	.	.
18	.	.	B=A*C	.	.	.	.
19	.	.	WRITE(B)	.	.	.	.
20	.	.	COMMIT	.	.	.	.
21	.	WRITE(C)	.	.	.	.	.
22	C=C-A	.	.	.	.	.	.
23	WRITE(C)	.	.	.	.	.	.
24	.	READ(C)	.	.	.	.	.
25	COMMIT	.	.	.	.	.	.
26	.	C=B*30	.	.	.	.	.
27	.	WRITE(C)	.	.	.	.	.
28	.	COMMIT	.	.	.	.	.

Rellenar también la siguiente tabla con el estado del servidor y comentarios de lo que está ocurriendo en cada secuencia de ejecución de las instrucciones de cada transacción en el servidor postgres.

Estado del servidor	Comentarios

Cuestión 16: ¿Cuál es el estado final de los datos A, B y C en la base de datos cuando han finalizado las tres transacciones? ¿Por qué?

Cuestión 17: Suponer que se produce una pérdida del cluster de datos y se procede a restaurar la instancia de la base de datos del punto 5. Realizar solamente la restauración (recovery) mediante el procedimiento descrito en el apartado 25.3.5 del manual (versión, 18) *"Continous Archiving and point-in-time recovery (PITR)*. Indicar el proceso seguido. ¿Cuál es el estado final de la base de datos? ¿Por qué?

Cuestión 18: A la vista de los resultados obtenidos en las cuestiones anteriores, ¿Qué tipo de sistema de recuperación tiene implementado postgreSQL? ¿Qué protocolo de gestión de la concurrencia tiene implementado? ¿Por qué? ¿Genera siempre planificaciones secuenciables? ¿Genera siempre planificaciones recuperables? ¿Tiene rollbacks en cascada? Justificar las respuestas.

Bibliografía

- Capítulo 9: System Information Functions and Operators, System Administration Functions.
- Capítulo 13: Concurrency Control.
- Capítulo 19: Server Configuration.
- Capítulo 25: Backup and Restore.
- Capítulo 27: Monitoring Database Activity.
- Capítulo 28: Reliability and the Write-Ahead log.